BIOGOALS.AGRI-ACODM_USER_MANUAL

User manual version: 1.0
Davide Carecci, 2025

Index

Preliminaries

The following user manual is intended to guide the other end-users of the *Biogoals.agri-AcoDM* tool. This tool is intended to be used by biomethane plant managers. It has the aim to provide a comprehensive physics-based model of the anaerobic co-digestion process, which can be used to:

- perform *off-line* optimisation of the plant design and/or diet;
- perform scenario analysis of different plant design and/or diets;
- mimic the behaviour of a real plant to conduct closed-loop tests over whatever kind of controller.

The high-fidelity agri-AcoDM model modelled in the **"agri-AcoDM.mo"** Modelica library is an extended and adapted version of the ADM1 model, that is the state-of-the-art in literature for the first-principle grey-box modelling of the anaerobic digestion process. Thus, in principle, it is enough comprehensive to be applied over many different reactor configurations, diet mixtures and operating conditions (especially if steady-states are simulated). However, the values of its parameters (especially the kinetic ones) are not "written in the rocks" (uncertain within certain min-max bounds), so that it can be relevant to conduct the so called 'calibration' of the model: the latter is conducted estimating the above-mentioned parameters by minimising the distance of the outputs' trajectories simulated/predicted by the model from the real available data (over a given open-loop simulating horizon for which the - possibly uncertain - states' initial conditions and inputs are known). Comparing preliminarily the model's performance against data, it is up to the user to decide whether to conduct a model calibration or stick with the current parameter values and consequent modelling mismatch.

When applying this tool from scratch on an existing plant, the following procedures shall be followed to adapt/calibrate the model over the specific case study:

1. Collect data of the plant (output measurements and diary of the corresponding input diet, plus feedstock's characterization, for example following the guidelines present in ...);
2. Create a new Modelica *model* inside the Modelica library name. Drag-and-drop the pre-defined *blocks* to recreate your plant. Load the *CombiTimeTables* that (convert from CSV/XLSX to TXT with "**generate_combi.ipynb**") contain information about the input feeding schedule (load of each co-feedstock in time) and the feedstock's compositions. Adjust the parameters of working volume and temperature;

3. Integrate the model and plot the results with "**Integrate.ipynb**", that exploits the "**modelica_integrator.py**" function. Compare the model outputs with the experimental data. If the model predictions (discarding the first window of data where results are more influenced by possible bad choices of the model initial conditions) are not sufficiently accurate, go ahead with this list;
4. Select the subset of interest of process parameters $\boldsymbol{\theta_P}$ (or also feedstocks' composition parameters if any is particularly uncertain/unknown) to conduct **parameter subset selection** (PSS), i.e. **sensitivity** and **collinearity analyses** (**practical identifiability analysis**) with the aid of "**Sensitivity_local_OAT.ipynb**" and "**Sensitivity_global_Sobol.ipynb**" notebooks. The subset $\boldsymbol{\theta_{PSS}}$ that contains the practically-identifiable entries of $\boldsymbol{\theta_P}$ is thus identified;
5. Estimate the values of $\boldsymbol{\theta_{PSS}}$ that minimises the distance of the model predictions from the data ($\boldsymbol{\theta^*_{PSS}}$) i.e. calibrate the model, with the aid of "**Parameter_estimation.ipynb**" that exploits the "**modelica_optimizer.py**" function;
6. Insert the new values of the parameters $\boldsymbol{\theta^*_{PSS}}$ inside the Modelica model and run again "**Sensitivity_local_OAT.ipynb**";
7. Load the results from "**Parameter_estimation.ipynb**" and the last run of "**Sensitivity_local_OAT.ipynb**" into "**Parameter_uncertainty_linear.ipynb**" to quantify and propagate linearly the parameter uncertainty related to $\boldsymbol{\theta^*_{PSS}}$ on the model outputs (95% confidence intervals of the model predictions assuming Gaussian distributions and linear relationship between parameters and outputs). Run "**Parameter_uncertainty_nonlinear.ipynb**" instead if a more realistic propagation is sought for (Monte Carlo-based, no approximation of linear relationship between parameters and outputs, but time consuming);
8. If the model fits the data with sufficient accuracy, reliably apply it to optimise *off-line* the input diet, with the aid of "**Optimization_diet.ipynb**" that exploits the "**modelica_optimizer.py**" function.
   See also the **"Model_calibration_pseudo_code.pdf"**.

Note: as described more in details inside the "**modelica_integrator.py**" function, the only way in which it is possible to set/modify some quantities inside the a Modelica model directly from Python and integrate, without FMU and/or other external libraries, is to declare inside the highest-level model called *model_name* (the one I want to simulate) the *param_dict.keys()* quantities. In other words, it is possible to set/modify only quantities declared as **parameter** and at the highest-level. To make the modification effective in lower-levels/sub-models, the user must set the *model_name* Modelica model in order to propagate the *param_dict.keys()* quantities to the values of the "true"/"actual" parameters' declarations (see the "**modelica_integrator.py**" documentation and the examples inside **"agri-AcoDM-mo"** for further details).

Note: in addition to the standard Python libraries present in **requirements.txt** the custom Python library **general_utils** is needed to run these codes. How to install it:

- if the user prefers a local installation in his/her computer: download the repository from GitHub, then run `pip install "path_to\general_utils"` in your Python virtual environment/Jupyter kernel;
- otherwise, install directly from GitHub running `pip install git+https://github.com/DaveCacci/general_utils.git` or `pip install git+https://<GITHUB_TOKEN>@github.com/DaveCacci/general_utils.git` if the repository is private (ask the creator for the token if not already available or deprecated). Alternatively, just add `git+https://github.com/DaveCacci/general_utils.git` to **requirements.txt** and install everything at once `pip install -r requirements.txt`.

File description

See the 'File naming' section of this manual for further details about the placeholders used to contract file names.

- ***"agri-AcoDM.mo"***: Modelica library that contains all the pre-defined *blocks* to built the custom *model* of the plant/case-study of interest. Among the blocks, the "main" one is the *Digester* block that contains all the physical equations of the process, i.e. an actual implementation of the agri-AcoDM model. See the 'Modelica library' section of this manual for further details.
- ***"Model_calibration_pseudo_code.pdf"***: pseudo-code in PDF file as further material to understand the full pipeline to be carried out for a proper model calibration.
- ***"CombiTimeTables/generate_combi.ipynb"***: Jupyter notebook to convert a CSV file to CombiTimeTable (TXT) format that can be read by Modelica.
  - All the other files present in this sub-folder are used in the models present inside **"agri-AcoDM.mo"**
- ***"Integration/Integrate.ipynb"***: Jupyter notebook to run one simulation/integration of the Modelica model of interest. If any, compares the model outputs with data and plots and computes model's performance metrics. Plots to allow for visual inspection. Main file outputs are:
  - ***"uit_real_R2_dynamic_output_results_07-01-2026_1145.csv***": CSV file that gives an example of the output trajectories' results extracted from the Modelica model's integration, created by running the notebook.
- ***"Sensitivity_ParameterSweep/Parameter_sweep.ipynb"***: Jupyter notebook to run multiple simulations/integrations of the Modelica model of interest, generating and setting multiple parameter samples. "Parameters" can be intended as every quantity that does not have a time-varying behaviour throughout the simulated horizon (e.g. process parameters, feedstock compositions, plant volume, operating temperature, constant input flow rate of each co-feedstock etc...). The example provided considers a subset of process parameters θ_P.
- ***"Sensitivity_ParameterSweep/Sensitivity_local_OAT.ipynb"***: Jupyter notebook to conduct linear once-at-a-time (OAT) sensitivity analysis (LSA) of the (measurable)

outputs to parameters (see the 'Sensitivity analysis (SA)' section of this manual for further details). Main file outputs are:

- ***"{modelname}Sensitivity_local_aa{absdeltap}{currentdate}{start}_{end}.xlsx"***: Excel file. Example of the local sensitivity indexes (expressed as "absolute-absolute"; trajectories in time before lumping by any unique metric). Computed at the given *abs_delta*. Same for files named with the "rr" ("relative-relative") string instead of "aa". If the "beforemod" string is also present in the filename, it is the file version without filtering with the available *off-line* measurement data points (used for uncertainty propagation purposes).
- ***"{modelname}Discontinuous_value_aa{absdeltap}{currentdate}{start}_{end}.xlsx"***: Excel file used to store the intermediate results of the sensitivity indexes' filtering with the available *off-line* measurement data points. Same for files named with the "rr" ("relative-relative") string instead of "aa".
- ***"{modelname}_inoculum.xlsx"***: Example of Excel file used to store the "inoculum" composition of each model evaluation. These are the values of the bulk state variables and are useful to initialise the models of batch tests, if any are present in the overall dataset.
- ***"{modelname}Output_nom{currentdate}{start}_{end}.xlsx"***: Excel file that contains the dynamic trajectories of the model's outputs, simulated with the "nominal" values of $\boldsymbol{\theta_P}$. If the "Sheet_Filter" string is present in the filename, it is the file version re-organised with each output's indexes stored in a separate Excel sheet and already filtered with the available *off-line* measurement data points.
- ***"{modelname}Output{deltap}{current_date}{start}_{end}.xlsx"***: Excel file that contains the dynamic trajectories of the model's outputs, simulated with the "nominal" values of $\boldsymbol{\theta_P}$ modified by "deltap". The same consideration reported above for the "Sheet_Filter" string in the filename holds.
- ***"{modelname}Collldx{currentdate}{outputnames}{start}_{end}.xlsx"***: Excel file. Example of the collinearity indexes, for each possible $\boldsymbol{\theta_P}$ subset, as computed by the notebook. If the "Stats" string is present in the filename, it is the file created by the PSS algorithm that combines the results of both the sensitivity and collinearity analyses to identify the best $\boldsymbol{\theta_{PSS}}$.
- ***"{modelname}StatsSorted_local{currentdate}{outputnames}{start}_{end}.xlsx***": Excel file. Example of the local sensitivity ranking as outputted by running the notebook.

- ***"Sensitivity_ParameterSweep/Sensitivity_global_Sobol.ipynb"***: Jupyter notebook to conduct global Sobol sensitivity analysis (GSA) of the (measurable) outputs to parameters (see the 'Sensitivity analysis (SA)' section of this manual for further details). Main file outputs are:
  - ***"{modelname}dynamic_output_results{currentdate}{start}_{end}.csv.gz"***: example of CSV.GZ file that stores the dynamic trajectories of all the model's outputs of interest, for each model run, i.e. sampled point of the uncertain $\boldsymbol{\theta_P}$ space.

    - ***"{modelname}scalar_output_results{currentdate}{start}_{end}.xlsx"***: example of Excel file that stores the values of $\boldsymbol{\theta_P}$ at each sample, for each model run. Other scalar (i.e. not time-dependent) quantities computed for each run are present too, if any (e.g. modelling errors with respect to data).
    - ***"{modelname}total_indices_sobol{current_date}.xlsx"***: example of Excel file that stores the computation of the total Sobol indexes (time trajectories for each output of interest). Same holds for the "first" (first-order Sobol indexes) and "second" (second-order Sobol indexes) strings present in the filename instead of "total".
    - Other files used/generated by this notebook are present in the folder. Their filename and related content are very similar to the ones already described above for the "local" sensitivity analysis.
- ***"Parameter_Estimation/Parameter_estimation.ipynb"***: Jupyter notebook to conduct model calibration i.e. uncertain parameter estimation to maximise the model's fitting capability of measured data (over a certain training set). See the 'Parameter estimation' section of this manual for further details. Main file outputs are:
    - ***"uit_real_R2+batch_april24_NM_optim_feval_01-12-2025_thetap.xlsx"***: Excel file that gives an example of the intermediate output expected running the notebook. Contains the list of both decision variables (process parameters in this case) and objective function value for each function evaluation performed by the optimiser.
    - ***"{modelname}Output_opt{currentdate}{start}-{end}.xlsx"***: Excel file that stores the model outputs evaluated with the optimised parameters $\boldsymbol{\theta^*_{PSS}}$. See the 'File naming' section of this manual for further details.
    - ***"{modelname}Residuals_opt{currentdate}{start}-{end}.xlsx"***: Excel file that stores the residuals (i.e. modeling errors) with respect to the data of the model outputs evaluated with the optimised parameters $\boldsymbol{\theta^*_{PSS}}$. See the 'File naming' section of this manual for further details.
- ***"Parameter_Estimation/Parameter_uncertainty_linear.ipynb"***: Jupyter notebook to quantify and propagate linearly to the model's outputs the uncertainty related to the parameters. In particular, the example computes the uncertainty related to the parameter estimates found after a run of the parameter estimation notebook (run for Carecci *et al.*, (2025)).
    - ***"uit_real_R2+batch_april24_Sensitivity_local_aa_0.01_25-11-2025_190113-100410.xlsx"***: this file is loaded as input in the notebook and it was created manually by merging together the results of the sensitivity analysis of different models (e.g. in the case there are data merged from multiple tests, see '1. Integration of activity/batch tests in parameter estimation' in the 'Parameter_estimation' section for further details).
      Main file outputs are:
    - ***"uit_real_R2_batch_april24_FIM_COV_sigma_01-12-2025.xlsx"***: example of Excel file that stores the main results of the notebook ($\mathbf{FIM}, \mathbf{COV}, \boldsymbol{\sigma_{\theta^*_{PSS}}}$).

    - ***"{modelname}StDev_Output{currentdate}{start}-{end}.xlsx"***: example of Excel file that stores the upper/lower bounds of the model's outputs confidence intervals.
- ***"Parameter_Estimation/Parameter_uncertainty_nonlinear.ipynb"***: Jupyter notebook to propagate to the model's outputs the uncertainty related to the parameters in a "locally-informed" nonlinear fashion. Uncertainty in $\boldsymbol{\theta}^*_{PSS}$ is propagated considering the nonlinearities present in the actual model, but sampling from a linearly-approximated multivariate distribution. A globally-informed correction of the uncertainty distributions is also provided via the "Importance Sampling" method. See bla bla for further details.
- ***"Diet_optimization/Optimization_diet.ipynb"***: Jupyter notebook to run an *off-line* (nonlinear constrained) optimisation of the feeding diet mixture exploiting the **"agri-AcoDM.mo"** (e.g. biomethane production rate maximisation under safety and regulating-authority constraints). Namely used to set the setpoints for the lower-level controllers. Main file outputs are:
    - ***"chile_DE_optim_feval_20.11.2025_diet.xlsx"***: Excel file that gives an example of the intermediate output expected running the notebook. Contains the list of both decision variables and objective function value for each function evaluation performed by the optimiser.

"Parameter_sweep.ipynb"

Standard libraries are imported as well as some custom functions present in a different or the same directory and, as mentioned above, some functions from the **general_utils** library. If a function present in another directory is needed, add `sys.path.insert(0, 'your_directory')`.

- User defines the input quantities required for the integration of the Modelica model with the "nominal" parameter values i.e. without fixing any different values externally (the values defined inside the Modelica model itself are considered). For example, *modelname* will help to maintain consistency between the output files created by the current code run/case study.
- The "**modelica_integrator.py**" function is called: parameter values and initial conditions have not been fixed differently. The output quantities of interest for extraction are specified and their trajectory is returned by the function.
- The "nominal" values of the parameters of interest for further sweep are extracted and scaled (user defined the list of names as declared in the highest-level Modelica model and the scales).
- The "Compute difference with respect to measurement data (if any)" is not compulsory. Since many model evaluations are going to be run, the evaluation of modelling error for each run is interesting in view of a future possibile parameter estimation (model calibration), performing some a-posteriori analysis on the relations between the parameters and outputs' variance. Both "*on-line*" and "*off-line*" data are loaded. The (model output; data) names couples are defined as input for the "**cumulative_error.py**"

function, that returns different error metrics primarily in the form of a dictionary that contains a key for each model outputs of interest.

- User defines the (min, max) bounds for parameter sweep and, optionally, their distribution as mean and standard deviation i.e. (μ, σ), for each parameter. The **generate_parameter_samples()** function is called to generate a matrix where each row contains a sample of the parameter space. See its documentation for further details. The *param_distrib* is used only for Monte Carlo-like sampling.
- *current_date* and *date_string* are also quantities that help to maintain consistency between the output files created by the current code run/case study, as well as *output_folder_path* that defines the path where the outputs of the parameter sweep will be saved at. **modelica_integrator()** is called for each parameter sample: their values is now override by the function exploiting the *param_dict* and *param_dict_scales* input arguments. Both *scalar_output_df* and *dynamic_output_df* are saved to external files: the former stores for each Modelica run the parameter values and the modelling errors (if computed), whereas the latter stores the whole outputs' trajectories (very heavy for memory usage, consider a different approach in future implementations. Indeed, now saved to a CSV.GZ i.e. zipped folder. See note in the notebook header/introduction).
- Some examples of possible post-processing/analysis of interest are present. User can use this section also loading old results stored externally, without running the multiple simulations again (very time consuming). For example:
  - plot the outputs' ensemble;
  - perform variance inflation factor (VIF) to see if the parameter samples suffer from collinearity (it can give an idea whether the dimensionality of the parameter sampling was enough to represent the parameter space or if it is better to increase it);
  - compute correlation metrics and plot correlation matrix between parameter values and modelling error (very preliminary form of sensitivity analysis). Plot a scatter between each parameter relative variations and the corresponding outputs relative variations (gives an idea of parameter global sensitivity and the type of non-linearities between parameter and outputs, although this is NOT a once-at-a-time evaluation. To do so, i.e. global (GSA) once-at-a-time (OAT) sensitivity analysis, implement 'Morris' sampling from the *SALib* library).

"Sensitivity_local_OAT.ipynb"

The "Nominal simulation..." section's description is very similar to what have been already reported above for **"Parameter_sweep.ipynb"**.

- User defines the *abs_deltap* list of relative parameter variations with respect to the "nominal" values extracted above (e.g. 1%, 10% etc...). Each parameter sample is generated modifying "once-at-a-time" (OAT) one nominal parameter value. A matrix where each row contains a sample of the parameter space is generated.

- Similarly to what described for **"Parameter_sweep.ipynb"**, a **modelica_integrator()** call is done for each entry of the above-mentioned matrix. The outputs' trajectories are stored in the *dynamic_output_df* and then saved to Excel file (one for each relative parameter variation of interest *deltap*).
- The "raw" outputs' trajectories are then re-organised in different Excel sheets and filtered between the two Timestamp of interest as defined by the user for the analysis. Note that both "nominal" and "*deltap*"'s Excel files have been previously saved, so that both *filter_and_save_nominal* and *filter_and_save_multiple* are called respectively.
- In the cells above, the "*off-line*" data have been loaded: it is important, if some "*off-line*" data are present in the dataset of interest, to filter the outputs' trajectories keeping only the simulated points where a data point exist! This holds true for both "*on-line*" and "*off-line*" data, but in general it is a critical aspect only for the latter as the frequency of the "*on-line*" data is supposed to match the *results_sample_interval* of the **modelica_integrator()** function.
- Perform the actual computation of the sensitivity matrices with finite-difference/central-different methods (*delta_method*) with the user-defined *formulation* ("absolute-absolute", "relative-relative", etc...) using the **compute_OAT_SI()** function. At this point, in the example, filter the sensitivity matrices on the Timestamps where the "*off-line*" data are present, for the "*off-line*" outputs of interest.
- To conclude the analysis, compute the sensitivity indexes (lumped quantities with respect to time for each parameter that is under investigation) and the relative ranking with the **sens_window()** function. If of interest (*collinearity* = True), the same function performs also the collinearity analysis following the Brun *et al,*. (2001) method. Later, the **parameter_choice()** function returns the suggested most practically identifiable subset of parameters ($\boldsymbol{\theta}_{PSS}$). If the collinearity analysis is not of interest, this choice is based purely on the sensitivity ranking/indexes magnitudes. Some meta-parameters shall be set by the user to guide this choice; see the notes on the notebook code for further details.
- Some examples of possible post-processing/analysis of interest are present:
  - collinearity index distribution and plot;
  - plot to verify the relation between parameters and outputs for the different *abs_deltap* i.e. relative output variation vs relative parameter variation, for each output-parameter couple.
    Note: for each function evaluation, the *final_states* i.e. the digestate composition is stored and later saved to Excel file. This is of interest when analysing parameter sensitivity across multiple Modelica models interconnected but practically separated inside the "**agri-AcoDM.mo**". See '1. Integration of activity/batch tests in parameter estimation' in the 'Parameter_estimation' section for further details.

"Sensitivity_global_Sobol.ipynb"

The "Nominal simulation..." and the "Compute difference...)"sections' descriptions are very similar to what have been already reported above for **"Parameter_sweep.ipynb"**.

- User defines the (min, max) bounds for parameter the "*saltelli*" sampling of the parameter space that is required to conduct Sobol analysis: the **generate_parameter_samples()** function is called with "*saltelli*" as *sampling_method* input argument to generate a matrix where each row contains a sample of the parameter space. Depending on the interest of the user to compute also the "second-order" Sobol indexes, the dimensionality of the matrix is defined considering the *alpha* meta-parameter defined by the user. See its documentation for further details.
- A model simulation for each parameter sample is performed very similarly to what have been already reported above for **"Parameter_sweep.ipynb"**.
- Sobol sensitivity matrices (indexes in time) are computed exploiting the *SALib* class of functions **"analyze_sobol()"**.
- Similarly to what have been described for **"Sensitivity_local_OAT.ipynb"**, the sensitivity ranking and the (linear) collinearity analysis can be performed.
- Some examples of possible post-processing/analysis of interest are present. User can use this section also loading old results stored externally, without running the multiple simulations again (very time consuming). Both the post-processing already described for **"Parameter_sweep.ipynb"** and **"Sensitivity_local_OAT.ipynb"**; in addition, first and second order Sobol indexes in time can be plotted, and, finally, an example to plot first-order indexes in time in an interactive plotting window in present (with "plotly" standard python package instead of "matplotlib")!!
  Note: the same 'Note' reported for **"Sensitivity_local_OAT.ipynb"** holds.

"Parameter_estimation.ipynb"

The "Nominal simulation..." and the "Compute difference..." sections' descriptions are very similar to what have been already reported above for **"Parameter_sweep.ipynb"**. However, the notebook reports an example of a complex case study of parameter estimation i.e. a case in which a Modelica model containing the batch tests (name of quantitites in general contains the "*_batch*" subscript) is initialised at a certain Timestamp with the digestate (i.e. inoculum) composition extracted from another Modelica model that simulates a reactor running in continuous operations (referred to as "*pilot*"). For this reason, the **modelica_integrator()** call for the batch tests model takes as input arguments both *x0_dict* (the dictionary of the "nominal" parameter values extracted from the *pilot* simulation) and *final_states_nom* (the dictionary containing the last values of digestate composition for the "nominal" simulation).

- User defines the (min, max) bounds as the constraints on the parameter search space for the optimisation algorithm that will be called later (generally expert-driven values and/or literature-suggested ones).
- The **modelica_optimizer()** class of functions is instantiated. See its documentation for details about the input arguments. In particular:
  - the first argument is a callable custom function to be evaluated at each exploration of the parameter space. It must return a dictionary of errors for the different outputs

of interest. In this example the **cumulative_error.py** function is used;
    - the *cost_args* and *integrator_kwargs* contain common and model-specific arguments if multiple Modelica models must be evaluated one after the other at each point of the parameter space explored by the optimization algorithm: the model-specific arguments are stored in the *model_chain* sub-dictionary. In addition, the (unique) *chain_mode* input argument defines if the model "i+1" shall be initialised from the final states of model "i" or from the states of the "first" model (for all "i" spanning the number of models that shall be evaluated).
- The user chooses the direct-search solver to be user. Up to now, the alternatives are DE (Differential Evolution) and NM (Nelder-Mead) algorithms. When dealing with very complex case studies as the one reported and a high number of parameters to be identified (generally higher than 4), it is suggested to start with some iterations of DE (e.g. 20-50), then refine with NM or 'polish" with a gradient-based algorithm (set *polish* = True for the **"differential_evolution()"** method). The optimisation is run and the results (for each Modelica run the parameter values i.e. decision variables and the modelling errors i.e. objective function value) are saved to an external Excel file.
- The "optimal" Modelica models' simulation is performed overriding the "nominal" values of the parameters with the "optimal" ones found so far (via *param_dict* and *param_dict_scales* input arguments; $\boldsymbol{\theta}^*_{PSS}$). The metrics what quantify model's performances over the "training" and "testing"/"validation" time windows and data sets are computed and saved externally, as well as the residuals and the outputs' trajectories of the "optimal" models' simulation for later usage in the **"Parameter_uncertainty_linear.ipynb"** notebook.
- Some examples of possible post-processing/analysis of interest are present. For example:
    - plot model "nominal" vs model "optimal" vs data trajectories;
    - compute the cumulative biomethane production for checking purposes (especially when dealing with batch tests and if the production rates are considered in the training set);
    - analyse the residuals. Extract from the outputs of the **cumulative_error()** function the dictionary of residuals and compute the error variance that can be later used in the **"Parameter_uncertainty_linear.ipynb"** notebook for the computation of the Fisher Information Matrix ($\mathbf{FIM}$). For each model output of interest, check the residuals' distribution, distance from 0 mean and normality with conventional statistical tests;
    - perform the same analysis described for **"Parameter_sweep.ipynb"** on the parameter points explored by the optimiser and the corresponding objective function values.
      Note: the outputs' trajectories for each Modelica model evaluation decided by the optimisation algorithm are not stored and saved in the current implementation. If memory usage are not of primary concern, consider saving them externally as done

in **"Parameter_sweep.ipynb"** and **"Sensitivity_global_Sobol.ipynb"** for later usage in **"Parameter_uncertainty_nonlinear.ipynb"**. see what for further details.

"Parameter_uncertainty_linear.ipynb"

The example reported in the notebook refers to the complex case study of parameter estimation in which data from both pilot operation and batch tests were considered (see the **"Parameter_estimation.ipynb"** description). This code is intended to be run AFTER the estimation of $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$ and sensitivity analysis (around $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$).

- The user defines the model outputs' names coherently to the ones of interest in **"Parameter_estimation.ipynb"**, and the related data name. The user defines the constant relative error of data (ideally taken from sensors' documentation and/or multiple observations of the same data; $\boldsymbol{\epsilon_{\tilde{y}}}$). The user reports as *parameter_dict* the optimal parameter values estimated via **"Parameter_estimation.ipynb"** ($\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$).
- The quantities already computed from parameter estimation and consequent sensitivity analysis around the estimated values are loaded. The quantities needed are:
  - the model's residuals computed from the "optimal" model simulation (i.e. with $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$) in **"Parameter_estimation.ipynb"**. If not computed previously, such quantities can be computed copying the appropriate code present in **"Parameter_estimation.ipynb"** inside the related section of this notebook;
  - the "optimal" output trajectories (i.e. computed with the "expected" estimated values of $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$);
  - the filtered and unfiltered (with the available *off-line* measurement data points) "absolute-absolute" sensitivity indexes computed at the $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$ *abs_deltap* = 1% (or less).
- From the "absolute-absolute" sensitivity indexes and residuals, the Fisher Information Matrix ($\mathbf{FIM}$), the covariance matrix ($\mathbf{COV}$) and the related parameter standard deviations ($\boldsymbol{\sigma}_{\boldsymbol{\theta}^*_{PSS}}$) are computed via the "**FIM.py**" function. The user sets the *error_variance, weighted, reg* and *epsilon* values (see the comments on the code for further details). T-test statistics is computed via the **"T_test.py"** function to verify the statistical significance of the estimates. Results can be saved to Excel file.
- From the "optimal" output trajectories and $\boldsymbol{\sigma}_{\boldsymbol{\theta}^*_{PSS}}$, the uncertainty in the parameter estimates is linearly propagated to the model's outputs via the **"lin_unc_prop.py"** function. The output's trajectories that define the *confidence_level*% confidence intervals (CI) are returned (default 95%, user can modify it). Results are saved to Excel file.
- To conclude the analysis, the $\mathbf{COV}$ is studied more in details to check for strong correlations between parameters (a posteriori "twin" of the collinearity analysis). If off-diagonal elements with high values do exist, it is suggested to: (i) consider to run again **"Parameter_estimation.ipynb"** selecting only one of the parameters that are strongly correlated with each other; (ii) be sure to propagate the uncertainty using the whole $\mathbf{COV}$ instead of its diagonal elements only ($diag(\mathbf{COV})$).

- The plot of the model "optimal" trajectories vs data can now be created together with the outputs' 95% CI.

"Parameter_uncertainty_nonlinear.ipynb"

The example reported in the notebook refers to the complex case study of parameter estimation in which data from both pilot operation and batch tests were considered (see the **"Parameter_estimation.ipynb"** description). This code is intended to be run AFTER the estimation of $\boldsymbol{\theta}^*_{PSS}$, sensitivity analysis (around $\boldsymbol{\theta}^*_{PSS}$) and computation of a linear approximation of the parameters' $\mathbf{COV}$. This notebook propagates such $\mathbf{COV}$ approximation in a non-linear way, via Monte Carlo sampling, and simulating multiple times the model to fully consider the non-linear and global effects in the parameter-output relations.

- The linear approximation of the $\mathbf{COV}$, namely computed from **"Parameter_uncertainty_nonlinear.ipynb"**, is loaded.
- Monte Carlo samples how to avoid negative values??. Check failed simulations.
- Multiple Modelica model runs (loop over *param_samples_valid*)
- In the code above the user can substitute the linearly-approximated $\mathbf{COV}$ with the one computed via the "Importance Sampling" method. This method requires...
  - from the parameter estimation...
  - from Monte Carlo propagation....

"Optimization_diet.ipynb"

The "Nominal simulation..." section's description is very similar to what have been already reported above for **"Parameter_sweep.ipynb"**. The main difference is that the error with respect to the data is not of interest for diet optimization: this code is intended to be run AFTER a complete and proper calibration of the high-fidelity model (the agri-AcoDM). This code performs the *off-line* nonlinear constrained optimization of the input diet mix. The cost function of interest in the example reported is the distance between the model's biomethane production rate and "a very high value": minimise such distance to, in turn, maximise the biomethane production rate. The Modelica model called for integration is designed to have an "initial"/"nominal" diet and consequent quasi-steady-state behaviour, a "transient" ramp period and a "final"/"optimal" diet, identified modifying the height of the ramp of each co-feedstock's input.

- User defines the upper and/or lower bounds of some output's quantities of interest to enforce constraints. In the example provided, the values of *VFA_ub, CODout_ub, HRT_ub, HRT_lb* (along the simulation, maximum TVFA value, maximum bulk total COD value, expert-driven maximum/minimum values of the reactor overall Hydraulic Retention Time (HRT)). The constraints are enforced via the *constraints* key of the *cost_args* input dict of the **modelica_optimizer()** class: here, the "**enforce_constraints()**" function is used, and it was designed to easily write constraints of *custom type* as nonlinear

functions of both the model's outputs (*custom*) and decision variables, i.e. parameters (*param_custom*). The constraints are enforced as penalties (**"exponential_penalty()"** function) to the cost function. See the documentation inside **modelica_optimizer.py** for further details.

- User defines the (min, max) bounds as the constraints on the parameter search space for the optimisation algorithm that will be called later (generally expert-driven values and/or literature-suggested ones).
- The **modelica_optimizer()** class of functions is instantiated. See its documentation and **"Parameter_estimation.ipynb"** for details about the input arguments. In this example the **min_error_constrained()** function is used as callable cost function to be minimized.
- In the example, the "**differential_evolution()**" method is called to effectively run the optimization (*polish* = True by default). The results (for each Modelica run the parameter values i.e. decision variables and the modelling errors i.e. objective function value) are saved to an external Excel file.
- The "optimal" Modelica models' simulation is performed overriding the "nominal" values of the parameters with the "optimal" ones found so far (via *param_dict* and *param_dict_scales* input arguments), i.e. the addition/subtraction with respect to the "initial"/"nominal" values of input flow rate for each of the co-feedstocks.
- In the example, the optimization is run with the *impulse* = True boolean parameter. This refers, inside the Modelica model, to the simulation of the manual and impulsive nature of the solid-rich co-feedstocks ($\mathbf{d}_{\mathbf{ref}}$). To extract the setpoints' trajectories ($\mathbf{y}_{\mathbf{ref}}$) that can be later used inside *Biogoals.Select* and *Biogoals.Twin*, the Modelica model is simulated again with the optimal diet transient found so far and *impulse* = False (real steady-state ("ss") behaviours expected over the period of "initial" and "final" diets' feeding). Thus, the setpoints' trajectories are saved to CSV and/or *CombiTimeTables* (the latter to run a closed-loop test directly simulating the bla bla model inside "**agri-AcoDM.mo**").

Additional notes

File naming

Saving of intermediate quantities of interest to CSV/XLSX files is required to smoothly move between different notebooks during the different stages elucidated in the 'Preliminaries' section. In the current implementation, input-output file naming is primarily "automatic" inside the majority of functions. The user can set some "meta-parameters" to better identity the different case-studies under investigation directly from the file names (and avoid undesired override):

- *modelname*: e.g. an abbreviation of the Modelica model name;
- *current_date*: the data in which the code/analysis is conducted;
- *start*: the initial timestamp of the model's output trajectories considered for the analysis (e.g. first row of the results extracted from the integration of the Modelica model ("*y_df_nom*"));

- *end*: the final timestamp of the model's output trajectories considered for the analysis (e.g. last row of the results extracted from the integration of the Modelica model ("*y_df_nom*"));
- *abs_deltap*: absolute value of the relative variation of $\boldsymbol{\theta_P}$;
- *deltap*: relative variation of $\boldsymbol{\theta_P}$.

Modelica library

ADD

Sensitivity analysis (SA)

1. Local SA (LSA): once-at-a-time (OAT)

1. Run multiple simulations, each one modifying just one parameter at-a-time, for each parameter variation magnitude of interest $\rightarrow$ 'raw' output results to Excel.
2. Reorganize the 'raw' Excel output files. Filter between two Timestamp values and, for off-line available-only outputs, extract the simulated values on the Timestamp values for which a measurement point do exist. Do this with **postprocess_OATsim_results** $\rightarrow$ save to Excel both output trajectories filtered and not filtered (*'_beforemod.xlsx'*) with such off-line measurement points.
3. Considering the Excel file with the outputs filtered with off-line measurement points, compute the sensitivity matrix with **compute_OAT_SI.py** for each output $\rightarrow$ save to Excel both "relative-relative" and "absolute-absolute" sensitivity matrices.
4. Compute the sensitivity indexes for each parameter concatenating together the information of all outputs using **sens_window.py** and reading the Excel file containing the sensitivity matrix (suggestion: use "relative-relative" sensitivity matrix). Time dimension is lumped with different metrics to create the sensitivity ranking. If the *collinearity* input is *true*, the main Excel file needed for the computation of the collinearity index is saved.
5. Use **parameter_choice.py** to complete the computation of the collinearity index and to use some heuristics to return $\boldsymbol{\theta_{PSS}}$ from the possible subsets (drivers: high sum of sensitivity index magnitudes and low collinearity).
6. Repeat up to point '3' after the estimation of $\boldsymbol{\theta_{PSS}}$ with **Parameter_estimation.ipynb**. Use the "absolute-absolute" sensitivity matrix to linearly quantify uncertainty (compute Fisher Information Matrix (**FIM.py**) that is a lower bound of the covariance matrix $\rightarrow$ compute $\boldsymbol{\sigma_{\theta_{PSS}}}$) and propagate it to the output trajectories with **Uncertainty_linear.ipynb**.

2. Global SA (GSA): Sobol

The Sobol's method is a variance-based method. Sobol indices quantify how much each input (and combinations of inputs) contributes to the output variance over the entire input domain.

The physical meaning of Sobol's indexes is the fraction of uncertainty in the output attributable to an input (alone or via interactions). It is not straight-forward to use Sobol's indexes in the computation of collinearity as in Brun *et al.*, (2001), as a matrix built from Sobol indices would not have eigenvalues with statistical or physical interpretation (loss of sign and direction; similar Sobol indices $\implies$ parameters contribute similar amounts to global variance, which does not imply indistinguishable effects). The resulting “collinearity” would not reflect identifiability, but only redundancy in variance contribution.
Thus, if *collinearity* = True in the "**sens_window()**" function, attention must be paid in the interpretation of the results, since the Brun *et al.*, (2001) method has not been formalised for global sensitivity analysis (GSA) such as Sobol! User must be careful when setting the *collinearity_range* in "**parameter_choice()**" function and interpreting results!
The Sobol's indexes cannot thus be consistently used to conduct a proper identifiability/collinearity analysis, but they can be used to compute the 95% CI of the $\boldsymbol{\theta}^*_{\boldsymbol{PSS}}$ estimates (see Juan C. Acosta-Pavas *et al.*, 2023).

If the number of parameter samples was set to be sufficiently high, and no weird trajectories were returned within the model runs (e.g. the presence of two different equilibrium within the parameter space under investigation Qian *et al.*, 2020), the "total" Sobol indexes should be the exact sum of the "first"-order and "second"-order (quantifying the interaction between parameters) Sobol indexes ($SI^{(i)}_{TOT} = SI^{(i)}_1 + \sum_{j \neq i} SI^{(ij)}_2$ for each index i and j of the $\boldsymbol{\theta_P}$ entries). For further details see Sobol analysis in SALib.

Parameter estimation

1. Integration of activity/batch tests in parameter estimation

In both **Sensitivity_global_Sobol.ipynb** and "**Parameter_estimation.ipynb**", the digestate composition is saved for each function evaluation of the continuous reactor (e.g. digester/'*pilot*') model. Those values are later used to initialise the inoculum composition for the batch test (e.g. BMP and activity) model. In this way, the effect of a parameter that characterises the same biological process and biomass in effectively transferred from the digester to the batch test, thus allowing the latter to contribute to the measurements' information content and to estimate such parameter with lower uncertainty. To ease this task during parameter estimation (and so for each function evaluation decided by the optimization algorithm), the Multi-Model chain mode in the "**modelica_optimizer.py**" class was introduced (see the class documentation and the '2. modelica_optimizer.py' paragraph for further details).

Such complication is handled in a different way in **Sensitivity_local_OAT.ipynb**: the notebook left as example is only for the *'pilot'* model. The same analysis was run for batch tests, as required by the case-study for consistency: the original code used for batch tests can be asked to the authors of the code. A note was added in the notebook on how to merge together the sensitivity matrices' results of *'pilot'* and batch tests into a unique Excel file (e.g. **"uit_real_R2+batch_april24_Sensitivity_local_aa_0.01_25-11-2025_190113-**

100410.xlsx") required as input for the further call of "**sens_window.py**" and "**FIM.py"** in "**Parameter_uncertainty_linear.ipynb**" (to quantify linearly the uncertainty).

2. modelica_optimizer.py

Structure:

- Single Model (only one Modelica model to be integrated):
  1. **Scale parameters**: Apply `param_scale_dict` to `param_values` → `param_dict`
  2. **Integrate**: Call `modelica_integrator(param_dict, **integrator_kwargs)` → `y_df`
  3. **Compute cost**: Call `cost_function(y_df, **cost_args)` → `cost_dict`
  4. **Enforce constraints**: If `constraints` in `cost_args`, add penalties to `cost_dict`
  5. **Record**: Append (params, costs, total_cost) to `iter_df`
  6. **Return**: `total_cost`
- Multi-Model Chain (via the *model_chain* argument of the *integrator_kwargs* input dictionary, more than one Modelica model can be integrated at each function evaluation. If *chain_mode* = 'parallel', the states' values at the final timestamp of the first model are used to initialise all the consequent models. If *chain_mode* = 'sequential', the states' values at the final timestamp of a model are used to initialise the consequent one.):
  1. **Scale parameters**: Same as single model
  2. **Integrate models sequentially**:
     - Model 0: Use `x0_dict` from `model_chain[0]`
     - Model 1+: Use final states from previous model (sequential) or first model (parallel)
     - Store outputs: `y_dfs = [y_df_0, y_df_1, ...]`
  3. **Compute costs per model**:
     - For each model i:
       - Merge common `cost_args` with `cost_args['model_chain'][i]`
       - Call `cost_function(y_dfs[i], **cost_args_i)` → `cost_dict_i`
       - Enforce constraints if specified
  4. **Merge costs**:
     - Detect duplicate keys across models
     - Add `_{i}` suffix only to duplicates (e.g., `pH_0`, `pH_1`)
     - Unique keys remain unchanged (e.g., `Qgas`, `Qch4_Ac_3`)
  5. **Record & return**: Same as single model
- Exception Handling: **Integration failure** (timeout, solver crash):
  - **Behavior**: Set all cost metrics to `1e12` (instead of `np.inf` to avoid optimizer overflow)
  - **Column consistency**:
    - If `iter_df` exists: Use existing column names (exclude params, `total_cost`)

- If `iter_df` empty:
  Use `final_output_names` or `outputs_extract_names` from config
- Multi-model: Merge output names from all models, apply same duplicate-suffix logic

Uncertainty quantification and propagation

1. Local/linear

After the selection of $\boldsymbol{\theta_{PSS}}$ as the identifiable subset of $\boldsymbol{\theta_p}$ and its *expected value* estimation, another run of **Sensitivity_local_OAT.ipynb** is run "centred" in the above-mentioned estimated values. From the results, the Fisher Information Matrix ($\mathbf{FIM}$) is computed exploiting the "absolute-absolute" sensitivity metrixes (Catenacci *et al.*, (2024)) as a lower bound approximation of the covariance matrix ($\mathbf{COV}$) of the parameter estimates. Actually, the $\mathbf{FIM}$ is a linear approximation of the real $\mathbf{COV}$, as a Gaussian Normal multivariate distribution is hypothesised for this computation (see the '2. Global/non-linear' paragraph for details). Then, the standard deviations ($\boldsymbol{\sigma_{\theta^*_{PSS}}}$) of the parameter estimates with respect to the average *expected value* estimated from **"Parameter_estimation.ipynb"** ($\boldsymbol{\theta^*_{PSS}}$) are computed as a first-order approximation, i.e. neglecting parameter correlations' effects and considering only the diagonal elements of the approximated $\mathbf{COV}$ (univariate Gaussian Normal distributions for each entry of $\boldsymbol{\theta_{PSS}}$). To correct the $\mathbf{COV}$, the model's residuals are computed exploiting the outputs of the **cumulative_error** function.

A constant relative error model (i.e. heteroscedastic data noise), for each output type (entry of $\mathbf{y}$), is considered to derive the standard deviations of data, used to weight the sensitivity matrix in the computation of the $\mathbf{FIM}$ (ideally taken from sensors' documentation and/or multiple observations of the same data; $\boldsymbol{\epsilon_{\tilde{y}}}$).

The linear uncertainty propagation is a strong approximation of the true global/non-linear propagation that considers the true/actual model effects in the parameter-output relations, especially if the model is strongly non-linear and many different operating conditions are "touched" during the simulation horizon. Moreover, if only the $\boldsymbol{\sigma_{\theta^*_{PSS}}}$ are considered (diagonal elements of the $\mathbf{COV}$), it is possible to commit a huge approximation, especially if parameter strongly change dependently one from the others (interactions). It is thus advised to propagate at least the "full" linear approximation of the $\mathbf{COV}$ as $Var(\tilde{y}) = \mathbf{SI}_{\tilde{y}} \times \mathbf{COV} \times \mathbf{SI}_{\tilde{y}}^T$ (where $\mathbf{SI}_{\tilde{y}}$ is the sensitivity matrix limited to the measurable output type $\tilde{y}$).

Note: the sensitivity matrices for each output considered in "**Parameter_estimation.ipynb**" filtered on the available *off-line* measurement data points is needed for the computation of the $\mathbf{FIM}$ (via the "**FIM.py**" function), whereas the "original" un-filtered nominal output trajectories and sensitivity matrices are needed for linear uncertainty propagation (via the "**lin_unc_prop.py**" function). For such tasks, it is needed that the quantities related to each output type are saved to a separate sheet of the Excel file.

two words about parameter scaling and issues in inverting the FIM if a parameter is unidentifiable (from local analysis). Better to use COV from Imp. Samp.?

2. Global/non-linear

Key Differences from Linear Propagation:

- Linear method ("**Parameter_uncertainty_linear.ipynb**")
  - Assumes: $\Delta y \approx SI_{aa} \times \Delta \boldsymbol{\theta}$ (first-order Taylor expansion)
  - Fast: single evaluation
  - Limitation: Ignores nonlinear output-parameter relationships
- Nonlinear method ("**Parameter_uncertainty_nonlinear.ipynb**")
  - Samples parameter space: $\boldsymbol{\theta_{PSS}} \sim \mathcal{N}(\boldsymbol{\theta^*_{PSS}}, \mathbf{COV})$
  - Runs full model simulations (but no multiple parameter estimations!)
  - Computes empirical percentiles from output distribution
  - Advantage: Captures full nonlinearity, asymmetric distributions
  - Cost: ~100-500 forward simulations

Important Caveat: Covariance Matrix Limitations. Critical consideration for interpretation: The parameter covariance matrix $\mathbf{COV}$ used for Monte Carlo sampling derived from linearized sensitivities around $\boldsymbol{\theta^*_{PSS}}$ means:

1. Local approximation: captures parameter correlations only in the neighborhood of $\boldsymbol{\theta^*_{PSS}}$.
2. Nonlinear reality: True parameter correlations may vary across parameter space
   - Compensation effects can change sign
   - Correlation strength may increase/decrease away from $\boldsymbol{\theta^*_{PSS}}$.
   Practical implication: Monte Carlo samples using this $\mathbf{COV}$ may not fully represent true nonlinear parameter correlations. Anyway, it still works reasonably well since:

- Most probable parameter values cluster near $\boldsymbol{\theta^*_{PSS}}$ (where $\mathbf{COV}$ is accurate)
- Captures dominant correlation structure from parameter estimation
- Much better than assuming independence ($diag(\mathbf{COV})$)
- Computationally feasible (vs. full MCMC/Profile Likelihood)

For truly accurate nonlinear correlations, the user would need:

- Markov Chain Monte Carlo (MCMC) with multiple parameter estimations, for formal posterior
- Profile Likelihood analysis (systematically map confidence regions)
- Bootstrap (repeated parameter estimation)
- Bayesian inference methods
  This is avoided here for computational reasons (parameter estimation is expensive!)
  Recommendation: treat results as "locally-informed nonlinear propagation" (better than pure linear, but not fully global). If the user requires a fully global uncertainty

quantification and propagation, and it has the possibility, the Bootstrap method only requires to:

- resample or perturb your data with sensor error distributions;
- run multiple times **"Parameter_estimation.ipynb"** for each sample of data;
- compute empirical percentiles and correlations.

The "Importance Sampling" method gives a globally-informed distribution of the $\boldsymbol{\theta}^*_{PSS}$' uncertainty, but "truly" global methods are MCMC, Profile Likelihood and Bootstrap. ADD on how to check, ESS etc...

Note: If you have already saved the necessary quantities (parameter sets, objective function values, and output trajectories) from your parameter estimation runs (e.g., Differential Evolution or Nelder-Mead) in `Parameter_estimation.ipynb`, you can load them and proceed directly to uncertainty propagation and importance sampling. This approach is often superior to Monte Carlo sampling from the local covariance, as it leverages actual function evaluations that explored the true likelihood surface and may better capture nonlinearities and multi-modality.

- If you have saved output trajectories for each parameter set, you can propagate confidence intervals on outputs without re-running forward simulations.
- If you only saved parameter sets and objective function values, you can use these for global parameter covariance estimation (importance sampling), but will need to re-run forward simulations to propagate output uncertainty.
- For propagation: filtering to the best N samples (lowest SSE) is recommended for efficiency and accuracy.
- For importance sampling: provide samples with SSE values spread out more widely across the likelihood surface - not just concentrated at the bottom of the valley. This gives more balanced weights and higher ESS.
  This workflow avoids unnecessary recomputation and makes full use of your optimization history for robust uncertainty quantification.

Alternatively, the 95% CI of the $\boldsymbol{\theta}^*_{PSS}$ estimates can be computed from the first-order Sobol indexes (see Juan C. Acosta-Pavas *et al.*, 2023), i.e. from the diagonal of the Global Sensitivity Information Matrix ($\mathbf{GSIM}$): however, this approach neglects the correlation between parameters, and I cannot use $\mathbf{GSIM}$ for the Monte Carlo sampling used to propagate non-linearly the uncertainty.

Further developments

- Evaluate both training and test metrics for each iteration during optimisation: when test metrics start to increase instead of decreasing, stop.
- Integrate the *Biogoals.TE-LP* results with the ones of "**Optimization_diet.ipynb**".

- Tip for efficiency when doing multiple simulations: use parallel processing if possible (e.g., *multiprocessing* or *joblib* standard Python libraries).